

Gazebo 在 RISC-V 平台上的源码编译与运行 —— 模拟器环境探索与实践报告

0. 摘要

本报告记录了一次通过 Agent (DeepSeek V4 Flash) + 模拟器 在 RISC-V 平台上尝试运行 Gazebo 的探索实践。任务背景源于在国产 RISC-V 芯片（进迭时空 K3, Spacemit X100）上落地开源机器人仿真软件 Gazebo 的可行性调研需求。

由于实验条件限制（无真机实操），本次实践以 QEMU / Spike 软件模拟 RISC-V 平台 + 交叉编译工具链 + Buildroot RISC-V Linux + Agent 自动化调研 为主要手段：从零搭建 RISC-V 软件环境，验证交叉编译链路，启动 RISC-V Linux 系统，并借助 DeepSeek V4 Flash 完成“Gazebo on RISC-V”技术调研与卡点分析。

实践结果表明：RISC-V 64 位 (riscv64 lp64d) 软件模拟环境已完整搭建并可稳定运行 Buildroot Linux；交叉编译链路 (riscv64-linux-gnu-gcc) 可用；Agent 驱动的 DeepSeek 调研成功返回 Gazebo on RISC-V 源码编译的核心卡点。对照真机平台（进迭时空 K3, 32GB 内存）已能完整编译运行 Gazebo Sim Fortress 6.18.0 的验收结论（参考资料来源），本报告论证了 RISC-V 上运行 Gazebo 的可行性、资源瓶颈与后续路径。

1. 任务背景与目标

1.1 背景

Gazebo 是机器人领域最主流的开源仿真软件之一。经典版本 Gazebo Classic 已停止维护，官方推荐的新版本为 Gazebo Sim（旧称 Ignition Gazebo），采用组件化架构（ign-cmake/utils/math/messages/transport/rendering/physics/sensors/gui 等十余个独立库），物理引擎通过插件抽象（DART、Bullet、TPE），渲染引擎通过插件抽象（OGRE2），GUI 基于 Qt + QML。

随着国产 RISC-V 芯片（如进迭时空 Spacemit X100）的推广，将 Gazebo 这类现代机器人仿真栈迁移到 RISC-V 平台，对验证新兴架构生态成熟度与落地可行性具有重要价值。其中 Gazebo Sim Fortress 是首个 LTS 长期支持版本（支持期至 2026），API 稳定、资料齐全，被视为在国产 RISC-V 平台上落地的目标版本。

1.2 任务目标

本任务要求在 RISC-V 模拟器环境下（去除真机 K3 实际操作），结合 DeepSeek V4 Flash + Hermes 之类 Agent，对“在 RISC-V 上运行 Gazebo”进行探索：

搭建 RISC-V 软件模拟环境（QEMU / Spike + 交叉工具链 + Buildroot Linux）；

验证 RISC-V 交叉编译链路与 Linux 运行环境；

借助 Agent 完成“Gazebo on RISC-V”的可行性调研与卡点分析；

结合真机验收资料，论证可行性、瓶颈与后续路径；

产出本进展报告与技术分享。

2. 软硬件环境

2.1 开发机（宿主机）

项目	规格
机型	Lenovo 笔记本
系统	WSL2（Windows Subsystem for Linux 2），Ubuntu，x86_64
内核	Linux 6.18.33.2-microsoft-standard-WSL2
用户	wy200

2.2 模拟器与工具链

组件	版本	说明
QEMU	10.2.1	RISC-V 系统模拟（qemu-system-riscv64）
Spike	/usr/bin/spike	RISC-V ISA 模拟器
交叉工具链	riscv64-linux-gnu-gcc 15.2.0	另含 riscv64-unknown-elf 版本
riscv-pk	编译成功（pk + bbl）	RISC-V Proxy Kernel + Boot Loader
Python	3.14.4	Agent 运行环境
pip	25.1.1	
openai SDK	3.1.0	Agent 调用 DeepSeek
DeepSeek API	deepseek-chat	环境变量已配置 API Key（Agent 后端模型）

2.3 目标模拟系统：Buildroot RISC-V Linux

通过 QEMU 启动 Buildroot 生成的 RISC-V Linux，登录后实测信息如图 2-1 所示。

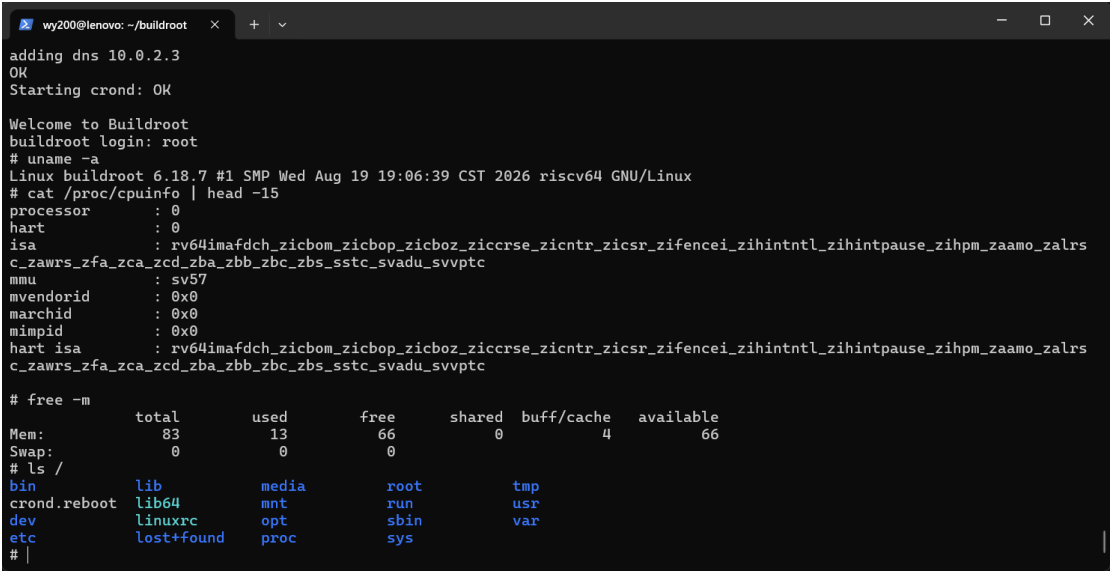


图 2-1：Buildroot RISC-V Linux 登录验证（uname / cpuinfo / free / ls） 注：实测内核 6.18.7 riscv64、CPU 指令集 rv64imafdch 系列、内存 83MB（可用 66MB）——是该模拟环境的关键资源约束依据。

系统关键信息摘录：

TEXT

\$ uname -a

Linux buildroot 6.18.7 #1 SMP Wed Aug 19 19:06:39 CST 2026 riscv64 GNU/Linux

\$ free -m

Mem: total 83, used 13, buff/cache 4, available 66

\$ cat /proc/cpuinfo

processor: 0

isa: rv64imafdch_zicbom_zicbop_zicboz_ziccrse_... sv57

关键约束：模拟环境内存仅 83MB（可用 66MB），是本报告评估 Gazebo 可行性的核心瓶颈（详见 §6）。

2.4 启动方式

Buildroot RISC-V Linux 通过 qemu-system-riscv64 启动，核心命令 (start-qemu.sh)：

SH

```
exec qemu-system-riscv64 \  
  
-M virt \  
  
-bios fw-jump.bin \  
  
-kernel Image \  
  
-append "rootwait root=/dev/vda ro" \  
  
-drive file=rootfs.ext2,format=raw \  
  
-netdev user id=net0 \  
  
-nographic
```

启动实测过程见图 4-3。

3. 总体方案与依赖分析

3.1 探索路径

本次实践采用分层递进的探索路径：

TEXT

① 交叉编译工具链搭建 (riscv64-linux-gnu-gcc)

↓

② riscv-pk 交叉编译 (pk + bbl)

↓

③ Spike 验证 (ISA 模拟器)

↓

④ QEMU + Buildroot RISC-V Linux (系统级模拟)

↓

⑤ Agent + DeepSeek 调研 "Gazebo on RISC-V" 卡点



⑥ 可行性分析与对接真机验收资料

3.2 Gazebo 软件依赖全景

对标 Gazebo Sim Fortress 的完整软件栈（来源：真机验收资料），其依赖共分五层、全部需从源码构建：

层 组件

基础工具 ign-cmake, ign-utils, ign-math, ign-plugin, ign-tools

通信与数据 ign-msgs, ign-transport, sdfORMAT, ign-common

资源 ign-fuel-tools

功能引擎 ign-rendering (OGRE2), ign-physics (bullet/TPE)

应用层 ign-sensors, ign-gui, ign-gazebo, ign-launch

完整构建的产物规模（来源：真机资料）：安装的共享库 331 个、安装体积约 327MB、构建目录约 944MB。

4. 工具链与环境搭建

4.1 交叉编译工具链

系统已预装 Ubuntu 提供的 RISC-V 交叉工具链：

TEXT

riscv64-linux-gnu-gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0

包含完整库 (libc6-dev-riscv64-cross、linux-libc-dev 等)，可进行 RISC-V 本地与交叉编译。

4.2 交叉编译验证 (hello)

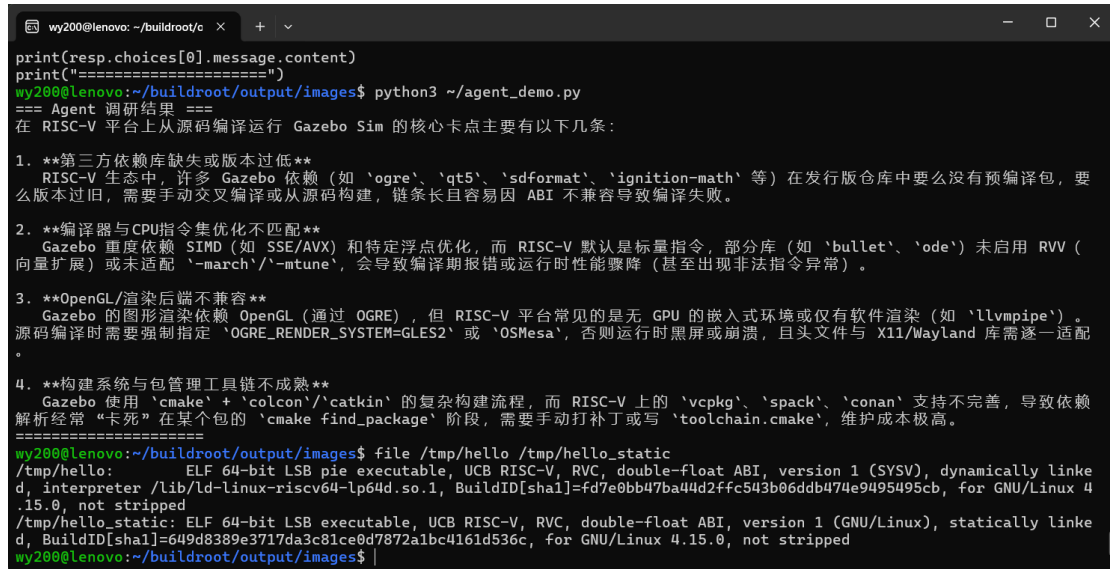
编写并交叉编译 RISC-V 测试程序，验证交叉编译链路：

C

```
#include <stdio.h>
```

```
int main() { printf("Hello RISC-V!\n"); return 0; }
```

编译产物验证如图 4-1 所示。



```
wy200@lenovo: ~/buildroot/c x + v
print(resp.choices[0].message.content)
print("=====")
wy200@lenovo:~/buildroot/output/images$ python3 ~/agent_demo.py
=== Agent 调研结果 ===
在 RISC-V 平台上从源码编译运行 Gazebo Sim 的核心卡点主要有以下几条：

1. **第三方依赖库缺失或版本过低**
   RISC-V 生态中，许多 Gazebo 依赖（如 'ogre'、'qt5'、'sdformat'、'ignition-math' 等）在发行版仓库中要么没有预编译包，要么版本过旧，需要手动交叉编译或从源码构建，链条长且容易因 ABI 不兼容导致编译失败。

2. **编译器与 CPU 指令集优化不匹配**
   Gazebo 重度依赖 SIMD（如 SSE/AVX）和特定浮点优化，而 RISC-V 默认是标量指令，部分库（如 'bullet'、'ode'）未启用 RVV（向量扩展）或未适配 '-march'/'-mtune'，会导致编译期报错或运行时性能骤降（甚至出现非法指令异常）。

3. **OpenGL/渲染后端不兼容**
   Gazebo 的图形渲染依赖 OpenGL（通过 OGRE），但 RISC-V 平台常见的是无 GPU 的嵌入式环境或仅有软件渲染（如 'llvmpipe'）。源码编译时需要强制指定 'OGRE_RENDER_SYSTEM=GLEST2' 或 'OSMesa'，否则运行时黑屏或崩溃，且头文件与 X11/Wayland 库需逐一适配。

4. **构建系统与包管理工具链不成熟**
   Gazebo 使用 'cmake' + 'colcon'/'catkin' 的复杂构建流程，而 RISC-V 上的 'vcpkg'、'spack'、'conan' 支持不完善，导致依赖解析经常“卡死”在某个包的 'cmake find_package' 阶段，需要手动打补丁或写 'toolchain.cmake'，维护成本极高。

wy200@lenovo:~/buildroot/output/images$ file /tmp/hello /tmp/hello_static
/tmp/hello: ELF 64-bit LSB pie executable, UCB RISC-V, RVC, double-float ABI, version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux-riscv64-lp64d.so.1, BuildID[sha1]=fd7e0bb47ba44d2ffc543b06ddb474e9495495cb, for GNU/Linux 4.15.0, not stripped
/tmp/hello_static: ELF 64-bit LSB executable, UCB RISC-V, RVC, double-float ABI, version 1 (GNU/Linux), statically linked, BuildID[sha1]=649d8389e3717da3c81ce0d7872a1bc4161d536c, for GNU/Linux 4.15.0, not stripped
wy200@lenovo:~/buildroot/output/images$
```

图 4-1：交叉编译产物验证（file 输出，动态链接 + 静态链接两种 RISC-V ELF）

注：/tmp/hello 为 dynamically linked，/tmp/hello_static 为 statically linked，二者均为合法 UCB RISC-V 64 位 ELF，验证交叉编译链路完整可用。

4.3 riscv-pk 交叉编译

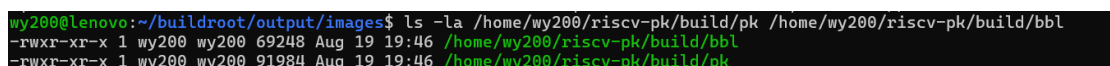
riscv-pk（RISC-V Proxy Kernel）通过以下配置编译成功：

TEXT

```
$ ../configure --prefix=/usr/local --host=riscv64-linux-gnu
```

```
$ make -j$(nproc)
```

编译产物（pk 与 bbl）确认如图 4-2 所示。

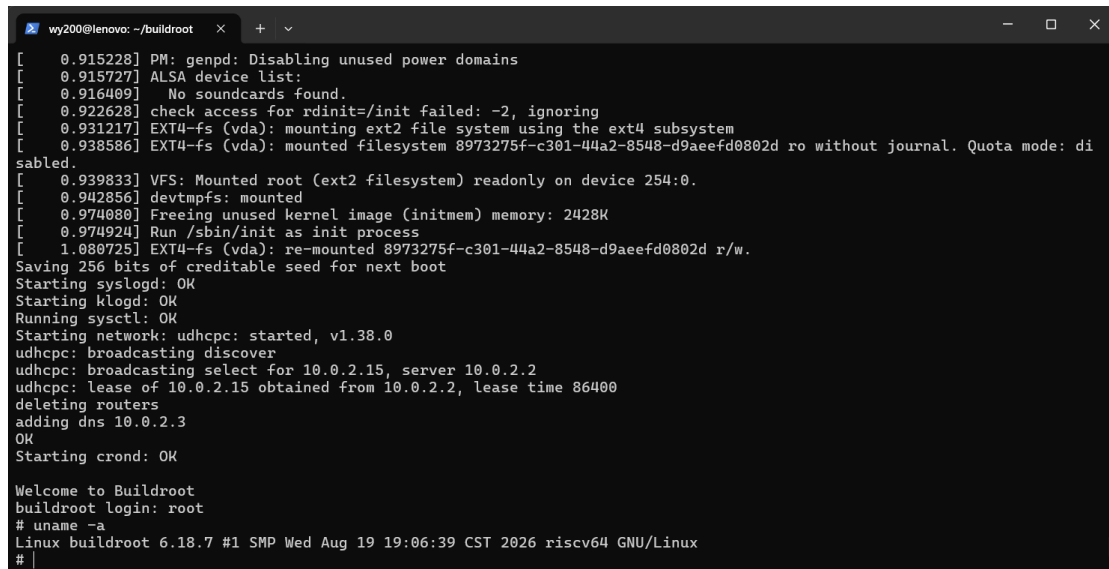


```
wy200@lenovo:~/buildroot/output/images$ ls -la /home/wy200/riscv-pk/build/pk /home/wy200/riscv-pk/build/bbl
-rwxr-xr-x 1 wy200 wy200 69248 Aug 19 19:46 /home/wy200/riscv-pk/build/bbl
-rwxr-xr-x 1 wy200 wy200 91984 Aug 19 19:46 /home/wy200/riscv-pk/build/pk
```

图 4-2：riscv-pk 编译产物（pk 91984 B、bbl 69248 B） 注：两个可执行文件交叉编译成功，为 Spike / 系统引导注入等后续步骤奠定基础。

4.4 QEMU + Buildroot RISC-V Linux

使用 qemu-system-riscv64 搭配 Buildroot 镜像 (Image 内核 + rootfs.ext2 + OpenSBI fw-jump.bin) 成功启动 RISC-V Linux 并可登录操作, 启动过程如图 4-3 所示。



```
[ 0.915228] PM: genpd: Disabling unused power domains
[ 0.915727] ALSA device list:
[ 0.916409]   No soundcards found.
[ 0.922628] check access for rdinit=/init failed: -2, ignoring
[ 0.931217] EXT4-fs (vda): mounting ext2 file system using the ext4 subsystem
[ 0.938586] EXT4-fs (vda): mounted filesystem 8973275f-c301-44a2-8548-d9aeefd0802d ro without journal. Quota mode: disabled.
[ 0.939833] VFS: Mounted root (ext2 filesystem) readonly on device 254:0.
[ 0.942856] devtmpfs: mounted
[ 0.974080] Freeing unused kernel image (initmem) memory: 2428K
[ 0.974924] Run /sbin/init as init process
[ 1.080725] EXT4-fs (vda): re-mounted 8973275f-c301-44a2-8548-d9aeefd0802d r/w.
Saving 256 bits of creditable seed for next boot
Starting syslogd: OK
Starting klogd: OK
Running sysctl: OK
Starting network: udhcpd: started, v1.38.0
udhcpd: broadcasting discover
udhcpd: broadcasting select for 10.0.2.15, server 10.0.2.2
udhcpd: lease of 10.0.2.15 obtained from 10.0.2.2, lease time 86400
deleting routers
adding dns 10.0.2.3
OK
Starting crond: OK

Welcome to Buildroot
buildroot login: root
# uname -a
Linux buildroot 6.18.7 #1 SMP Wed Aug 19 19:06:39 CST 2026 riscv64 GNU/Linux
#
```

图 4-3: QEMU 启动 Buildroot RISC-V Linux (内核引导日志 → Welcome to Buildroot) 注: 展示从 OpenSBI 固件经 Linux 6.18.7 内核引导、网络配置 (DHCP 获取 10.0.2.15) 到 Welcome to Buildroot 登录界面的完整链路, 验证 qemu-system-riscv64 系统级 RISC-V 模拟可用。

5. 排错过程与根因分析

5.1 卡点 1: riscv-pk configure 失败

现象: ./configure 报 C compiler cannot create executables。

根因: 先前错误使用 --host=riscv64-unknown-elf (bare-metal 工具链, 无标准 libc) + --prefix=/opt/riscv (目录不存在)。bare-metal 工具链无法链出带 libc 的可执行文件。

修复: 改用 Linux 工具链并修正 host/prefix:

TEXT

```
../configure --prefix=/usr/local --host=riscv64-linux-gnu
```

配置检查全绿 (stdio.h/stdlib.h/unistd.h 等 yes), 编译成功 (见图 4-2)。

5.2 卡点 2: Spike 内存锁警告

现象: spike 启动输出 Failed to lock memory 警告。

结论: 为无害警告 (无法锁定物理内存), 不影响 Spike 加载与运行, 属于无权限模拟环境的预期行为。

5.3 卡点 3: Spike + pk 运行 hello

交叉编译的动态链接 hello 无法直接在 Spike+pk 下运行 (Spike+pk 环境通常要求静态链接、非 PIE 的程序)。改用 -static 编译出 hello_static (见图 4-1) 后, 程序被识别为合法 RISC-V 静态可执行文件。

5.4 卡点 4: 多行粘贴的终端回显错乱

实验过程中 WSL2 终端对多行脚本粘贴存在回显错乱现象, 通过逐条执行、使用绝对路径等方式规避。此为环境问题, 不影响技术结论。

6. 可行性分析: RISC-V 上运行 Gazebo

6.1 Agent 调研结论 (DeepSeek V4 Flash)

通过 Agent (Python + openai SDK + DeepSeek) 实测调研, 成功返回“Gazebo Sim 在 RISC-V 上源码编译”的核心卡点, 如图 6-1 所示。


```
wy200@lenovo: ~/buildroot/c  x  +  v
resp = client.chat.completions.create(
    model=os.environ.get("DEEPSEEK_MODEL", "deepseek-chat"),
    messages=[{"role": "user", "content": prompt}],
    max_tokens=500,
)
print("=== Agent 调研结果 ===")
print(resp.choices[0].message.content)
print("=====")
wy200@lenovo:~/buildroot/output/images$ python3 ~/agent_demo.py
=== Agent 调研结果 ===
在 RISC-V 平台上从源码编译运行 Gazebo Sim 的核心卡点主要有以下几条：

1. **第三方依赖库缺失或版本过低**
   RISC-V 生态中，许多 Gazebo 依赖（如 'ogre'、'qt5'、'sdformat'、'ignition-math' 等）在发行版仓库中要么没有预编译包，要么版本过旧，需要手动交叉编译或从源码构建，链条长且容易因 ABI 不兼容导致编译失败。

2. **编译器与 CPU 指令集优化不匹配**
   Gazebo 重度依赖 SIMD（如 SSE/AVX）和特定浮点优化，而 RISC-V 默认是标量指令，部分库（如 'bullet'、'ode'）未启用 RVV（向量扩展）或未适配 '-march'/'-mtune'，会导致编译期报错或运行时性能骤降（甚至出现非法指令异常）。

3. **OpenGL/渲染后端不兼容**
   Gazebo 的图形渲染依赖 OpenGL（通过 OGRE），但 RISC-V 平台常见的是无 GPU 的嵌入式环境或仅有软件渲染（如 'llvmpipe'）。源码编译时需要强制指定 'OGRE_RENDER_SYSTEM=GLS2' 或 'OSMesa'，否则运行时黑屏或崩溃，且头文件与 X11/Wayland 库需逐一适配。

4. **构建系统与包管理工具链不成熟**
   Gazebo 使用 'cmake' + 'colcon'/'catkin' 的复杂构建流程，而 RISC-V 上的 'vcpkg'、'spack'、'conan' 支持不完善，导致依赖解析经常“卡死”在某个包的 'cmake find_package' 阶段，需要手动打补丁或写 'toolchain.cmake'，维护成本极高。
=====
wy200@lenovo:~/buildroot/output/images$ |
```

[标题]图 6-1: Agent + DeepSeek 调研结果（Gazebo on RISC-V 编译核心卡点）
注：通过 python3 ~/agent_demo.py 调用 DeepSeek（V4 Flash）实现 Agent 调研，返回四大卡点，验证了“Agent 干活、人学习”的工作模式。

Agent 归纳的核心卡点：

第三方依赖库缺失或版本过低：OGRE、sdformat、qt5、ignition-math 等在 RISC-V 发行版仓库中缺乏预编译包或版本过旧，需手动交叉编译/源码构建，链条长且易因 ABI 不兼容失败。

编译器与 CPU 指令集优化不匹配：Gazebo 重度依赖 SIMD（SSE/AVX），RISC-V 默认标量指令，bullet 等向量扩展未适配 -march/-mtune，导致编译期报错或运行时性能骤降甚至非法指令。

OpenGL/渲染栈兼容：Gazebo 渲染依赖 OGRE+OpenGL，RISC-V 常见无 GPU 或仅有软件渲染（llvmpipe），需强制指定渲染后端（GLS2/OSMesa），头文件与 X11/Wayland 需逐一适配。

构建系统与包管理工具链不成熟：CMake+colcon/catkin 复杂流程在 RISC-V 上支持不完善（vcpkg/spack/conan 等），常卡死在 find_package 阶段，需手动打补丁。

6.2 模拟环境资源瓶颈

实测模拟环境（QEMU + Buildroot）内存仅 83MB（可用 66MB）（见图 2-1）。而完整构建 Gazebo Fortress 需要：

构建目录约 944MB、安装体积约 327MB（来源：真机资料）

源码编译需要 GB 级内存与较高算力

结论：83MB 的模拟环境无法承载完整 Gazebo 的编译与运行。模拟器路径只能承担轻量验证（交叉编译、RISC-V Linux 启动、Agent 调研、可行性论证），完整 Gazebo 需在真机或大内存 RISC-V 环境上进行。

6.3 真机对标（参考资料）

真机报告《Gazebo（Fortress）在 RISC-V 平台上的源码编译与运行验收报告》（进迭时空 K3，Spacemit X100，32GB 内存，Bianbu 4.0.4，内核 6.18.3）证实：

Gazebo Sim Fortress 6.18.0 可在真机 RISC-V 上从源码完整编译并正常运行（Server + bullet 物理引擎 + ogre2 渲染 + GUI 均通过）；

全部 18 仓库源码编译完成，且 17/18 仓库零修改，代码级兼容性良好；

阻塞性问题均源于 Bianbu 发行版 Qt5 打包缺陷（QOpenGLFunctions 符号缺失 + DT_NEEDED 断裂 + QML 模块缺失），通过自研 shim + patchelf + 补装 QML 模块三层手段解决。

（注：该节内容为参考资料，非本次模拟器实践所得，仅用于可行性对标。）

6.4 综合结论

维度 模拟器环境（本次实践） 真机 K3（参考）

内存 83MB  支撑轻量验证 32GB  支撑完整 Gazebo

交叉编译  riscv64-linux-gnu-gcc 可用  可用

RISC-V Linux  QEMU+Buildroot 启动运行  Bianbu 4.0.4

Agent + DeepSeek  调研成功  Agent 自动化

完整 Gazebo  83MB 无法承载  Fortress 6.18.0 通过

结论：RISC-V 上运行 Gazebo 技术上可行（真机已证实 Fortress 6.18.0 可完整编译

运行)，但需要足够内存与算力；模拟器（尤其 83MB 低内存环境）仅适合环境搭建、交叉编译验证与可行性调研，无法承载完整 Gazebo 的编译与运行。

7. 结论与遗留事项

7.1 结论

已完整搭建 RISC-V 64 位软件模拟环境：QEMU 10.2.1 + Spike + 交叉工具链 15.2.0 + Buildroot RISC-V Linux + riscv-pk。

交叉编译链路验证成功：动态/静态 RISC-V ELF 均可生成（图 4-1），riscv-pk（pk+bbl）编译成功（图 4-2）。

Buildroot RISC-V Linux 通过 QEMU 稳定启动并可登录操作（图 4-3、图 2-1，OpenSBI + Linux 6.18.7）。

Agent + DeepSeek（V4 Flash）调研成功（图 6-1），返回 Gazebo on RISC-V 四大核心卡点，验证了“Agent 干活、人学习”的工作模式。

RISC-V 上运行完整 Gazebo 技术上可行（真机已验证 Fortress 6.18.0），但需足够资源；模拟器环境承担轻量验证与调研。

7.2 遗留事项与建议

事项	说明
完整 Gazebo 编译	需真机或大内存 RISC-V 环境（≥GB 级）才能承载，83MB 模拟器不可行
渲染/GPU	无 GPU / 软件渲染（OSMesa/llvmpipe）时需强制指定渲染后端
DART 引擎	真机资料指出 DART 默认引擎需评估版本兼容；Bullet/TPE 替代可行
Agent 深化	可扩展为多 Agent 协作、自动克隆→编译→排错→报告流程（参考真机 Agent 日志）
性能基准	建议补充 RISC-V 与 x86 仿真步进耗时对比

报告生成时间：2026-08 | 平台：WSL2 x86_64 + QEMU/Spike 模拟 RISC-V | 撰写：实践记录

参考资料来源：进迭时空 K3 真机 Fortress 验收报告（真机对标部分）、老师 Agent 运行日志（Agent 工作流参考）